CORRIGÉ · PROBLÈME · ANNALES SPÉ

Sous-graphe le plus dense

Énoncé

1. Réseaux, chemins augmentants et plus petite bordure

Langage C (ou pseudo-code de même niveau). Un réseau $H = (W, F, \mu, s, t)$: sommets numérotés de 0 à $n+1$, source $s = 0$, puits $t = n+1$, multiplicité $\mu(e) \in \mathbb{N}^*$ pour chaque arc $e \in F$. Fichier `network.h` :

```
#ifndef NETWORK_H
#define NETWORK_H
const unsigned int NMAX = 102;
struct network_s { unsigned int n; unsigned mu[NMAX][NMAX]; };
typedef struct network_s network;
void nw_init(unsigned int n);
void nw_add(int u, int v);
void nw_remove(int u, int v);
void nw_disconnect(void);
void _arr_init(int *T);
bool _bfs_tree(void);
void _residue(void);
#endif
```

Fichier `network.c` : `network H; int A[NMAX];` ; `H.mu[u][v]` vaut $\mu((u,v))$, ou 0 si (u,v) n'est pas un arc.

1. Rôle des lignes `#ifndef`, `#define` et `#endif`.
2. Comment faire interagir `network.h` et `network.c` ?
3. `void nw_init(unsigned int n)` : précondition $n < n_{\max}$ vérifiée par une assertion ; `H` devient le réseau vide à $n+2$ sommets.
4. `void nw_add(int u, int v)` : incrémente $\mu[u][v]$.
5. `void nw_remove(int u, int v)` : retire une occurrence, avec une assertion si l'arc n'existe pas.
6. `void _arr_init(int *T)` : met les $n+2$ cases de `T` à -1.
7. `bool _bfs_tree(void)` : calcule dans `A` une arborescence de plus courts chemins (parcours en largeur depuis s) : `A[s] = s`, `A[v]` = prédécesseur pour v accessible, -1 sinon ; renvoie `true` si t est accessible.

Un ensemble Π de k chemins de s à t est réalisable si chaque arc e est utilisé au plus $\mu(e)$ fois.

Le réseau résiduel $\mathrm{Res}(H, \Pi)$ s'obtient en retirant, pour chaque arc (u, v) de chaque chemin, une occurrence de (u, v) et en ajoutant une occurrence de (v, u).

1. `void _residue(void)` : remplace `H` par $\mathrm{Res}(H, \{\alpha\})$, où α est le chemin de s à t dans l'arborescence `A`.
2. Après k appels successifs (chemins $\alpha_1, \dots, \alpha_k$), montrer qu'il existe un ensemble réalisable Π de k chemins tel que le réseau obtenu soit $\mathrm{Res}(H, \Pi)$.
3. `void nw_disconnect(void)` : tant qu'il existe un chemin de s à t, remplacer H par le résiduel le long d'un plus court chemin ; à la fin, `A` est une arborescence de racine s n'atteignant pas t.
4. Montrer que `nw_disconnect` termine (variant inspiré de 9).

Bordure ∂S : arcs $(u, v) \in F$ avec $u \in S$, $v \notin S$; $\mu(\partial S) = \sum_{e \in \partial S} \mu(e)$. Un entourage est un $S \subseteq W$ avec $s \in S$, $t \notin S$.

1. Pour Π réalisable de k chemins et S entourage : $k \leqslant \mu(\partial S)$.

On fixe Π_0 réalisable de k_0 chemins tel que $\mathrm{Res}(H, \Pi_0)$ n'ait plus de chemin de s à t, et $S_0 =$ sommets accessibles depuis s dans $\mathrm{Res}(H, \Pi_0)$.

1. Pour tout arc (u, v) avec $u \in S_0$, $v \notin S_0$: toutes ses $\mu(e)$ occurrences sont utilisées par Π_0.
2. Aucune occurrence d'un arc (v, u) avec $v \notin S_0$, $u \in S_0$ n'est utilisée par Π_0.
3. Montrer $k_0 = \mu(\partial S_0)$.
4. Montrer $k_0 = \sigma(H)$ (plus petite bordure d'un entourage) et que, après `nw_disconnect`, $\{v : A[v] \geqslant 0\}$ est un entourage de plus petite bordure.

2. Sous-graphe le plus dense : l'algorithme de Goldberg

Graphe non orienté $G = (V, E)$ à sommets $1, \dots, n$, listes d'adjacence doublement chaînées : `struct link_s { int node; struct link_s *prev, *next; }` ; `struct graph_s { unsigned int n; link **neighbours; }`, tableau de longueur $n + 1$ (case 0 inutilisée) ; chaque arête est stockée comme deux arcs. Densité $\delta(G) = \frac{m}{n}$; $\rho(G) = \max_{X \neq \emptyset} \delta(G_X)$, où G_X est le sous-graphe induit par X.

1. `graph *gr_create(unsigned int n)` : graphe vide à n sommets.
2. `void gr_add(graph *G, int u, int v)` : insère l'arête $\{u, v\}$.
3. `int degree(graph *G, int v)`.
4. `int edges(graph *G)` : nombre d'arêtes.
5. `double density(graph *G)`.

Réseau associé à G (n sommets, m arêtes) et à un entier r : $W = V \cup \{s,t\}$; arc (s,v) de multiplicité mn^2 pour tout v ; arcs (u,v) et (v,u) de multiplicité n^2 pour toute arête ; arc (u,t) de multiplicité $mn^2 - n^2 \deg_G(u) + 2r$ pour tout u. Question : a-t-on $\rho(G) > \frac{r}{n^2}$?

1. `void reduce(graph *G, int r)` : place dans `H` le réseau associé.
2. Montrer $\delta(G_X) = \dfrac{\sum_{v\in X} \deg_G(v) - \mathrm{card}\{\{u,v\} \in E : u \in X,\ v \notin X\}}{2|X|}$.
3. Pour $S = \{s\} \cup X$: $\mu(\partial S) = mn^3 + 2|X|(r - n^2\delta(G_X))$, et $\mu(\partial\{s\}) = mn^3$.
4. Montrer : $\sigma(H) = mn^3 \iff \rho(G) \leqslant \frac{r}{n^2}$ et $\sigma(H) < mn^3 \iff \rho(G) > \frac{r}{n^2}$.
5. `bool oracle(graph *G, int r)` : renvoie vrai si et seulement si $\rho(G) > \frac{r}{n^2}$. Justifier.

$\Delta = \{(m',n') : 0 \leqslant m' \leqslant m,\ 1 \leqslant n' \leqslant n\}$.

1. Si $\frac{m_1}{n_1} \neq \frac{m_2}{n_2}$ (couples de Δ), alors $\left|\frac{m_1}{n_1} - \frac{m_2}{n_2}\right| \geqslant \frac{1}{n^2}$.
2. S'il n'existe aucun sous-graphe induit de densité $\geqslant \delta(G_X) + \frac{1}{n^2}$, alors $\delta(G_X) = \rho(G)$.
3. `void binary_search(graph *G)` : modifie `A` pour que $X = \{v \in V : A[v] \geqslant 0\}$ vérifie $\delta(G_X) = \rho(G)$. Justifier.
4. Comment faire interagir `densest_subgraph.c` avec `network.h` et `network.c` ?

3. Algorithme glouton de Charikar

Algorithme de Charikar : $\Gamma \leftarrow G$, $G_n \leftarrow G$; pour i de n à 2 : si $\delta(G_i) > \delta(\Gamma)$ alors $\Gamma \leftarrow G_i$; on retire de G_i un sommet v_i de degré minimal (et ses arêtes) pour former G_{i-1} ; on renvoie Γ.

Une orientation de G choisit un sens pour chaque arête ; $\deg^-(v)$ est le degré entrant. $\widehat{G_0}$ oriente l'arête $\{v_i, v_{i'}\}$ en $(v_{\min(i,i')}, v_{\max(i,i')})$, c'est-à-dire vers le sommet dont le retrait l'a supprimée.

1. Montrer $\delta(G) \leqslant \max_v \deg^-_{\widehat{G}}(v)$ pour toute orientation $\widehat{G}$ de G.
2. Montrer $\deg^-_{\widehat{G_0}}(v_i) \leqslant 2\delta(G_i)$ et en déduire un facteur d'approximation de l'algorithme.
3. Détailler une structure de données qui exécute l'algorithme en $O(n+m)$ (suggestion : une table des sommets de degré d pour chaque $d \in [\![0,n]\!]$).

Décoder l'énoncé

Ce que l'énoncé te donne.

- Les variables `H` et `A` sont globales : les fonctions n'ont presque pas d'arguments.
- Multiplicité = nombre de copies d'un arc ; résiduel = on « retourne » une copie de chaque arc emprunté.
- Dans le réseau associé, la source est 0, les sommets du graphe $1, \dots, n$, le puits $n+1$: la numérotation colle avec celle du réseau.
- Le sommet retiré à l'étape i a un degré minimal dans G_i, donc au plus la moyenne $\frac{2m_i}{i}$.

Ce qu'on te demande. Construire des chemins réalisables jusqu'à déconnecter s de t, puis prouver que leur nombre est la plus petite bordure (forme du théorème flot max / coupe min).

Corrigé

Question 1

À la première inclusion, `NETWORK_H` n'est pas défini : `#ifndef NETWORK_H` laisse passer le contenu, et `#define NETWORK_H` le définit. À toute inclusion suivante dans le même fichier compilé, `#ifndef` est faux et le préprocesseur saute tout jusqu'au `#endif`. Cela évite les doubles définitions (de `struct network_s`, du `typedef`…), qui provoqueraient des erreurs de compilation.

Question 2

- `network.c` commence par `#include "network.h"` (guillemets : fichier local), qui lui donne la structure, la constante et les prototypes.
- Tout autre fichier qui utilise le réseau inclut aussi `network.h`.
- On compile séparément (`gcc -c network.c` donne `network.o`), puis on lie les objets : `gcc network.o main.o -o prog`.

Remarques de rigueur : en C (contrairement au C++), `const unsigned int NMAX` n'est pas une expression constante, donc ne peut pas dimensionner un tableau dans une `struct`, et une définition de variable dans un en-tête inclus par plusieurs fichiers provoque une double définition à l'édition de liens. On écrirait plutôt `#define NMAX 102`. De même, il faut `#include <stdbool.h>` pour le type `bool`.

Question 3

```
void nw_init(unsigned int n) {
    assert(n < NMAX);              /* précondition (voir remarque) */
    H.n = n;
    for (unsigned int u = 0; u < n + 2; u++)
        for (unsigned int v = 0; v < n + 2; v++)
            H.mu[u][v] = 0;        /* aucun arc */
}
```

Remarque : les indices vont jusqu'à $n+1$; pour ne pas déborder, il faudrait en fait $n+2 \leqslant n_{\max}$ (soit `assert(n + 2 <= NMAX)`). La précondition de l'énoncé, $n < n_{\max}$, est un peu trop faible.

Question 4

```
void nw_add(int u, int v) {
    H.mu[u][v] = H.mu[u][v] + 1;   /* une occurrence de plus de l'arc (u,v) */
}
```

Question 5

```
void nw_remove(int u, int v) {
    assert(H.mu[u][v] > 0);        /* l'arc doit exister */
    H.mu[u][v] = H.mu[u][v] - 1;
}
```

L'assertion est indispensable : `mu` est non signé, $0-1$ donnerait un très grand entier.

Question 6

```
void _arr_init(int *T) {
    for (unsigned int v = 0; v < H.n + 2; v++)
        T[v] = -1;
}
```

Question 7

```
bool _bfs_tree(void) {
    int N = H.n + 2;              /* nombre de sommets */
    int file[NMAX];               /* chaque sommet est enfilé au plus une fois */
    int debut = 0, fin = 0;
    _arr_init(A);
    A[0] = 0;                     /* la source est sa propre racine */
    file[fin++] = 0;
    while (debut < fin) {
        int u = file[debut++];    /* on défile */
        for (int v = 0; v < N; v++)
            if (H.mu[u][v] > 0 && A[v] == -1) {   /* arc (u,v), v pas encore vu */
                A[v] = u;
                file[fin++] = v;
            }
    }
    return A[N - 1] != -1;        /* le puits t = n+1 est-il atteint ? */
}
```

Un sommet reçoit son prédécesseur à sa découverte et n'est enfilé qu'une fois, donc la file de taille $n+2$ suffit. Le parcours en largeur découvre les sommets par distance croissante : l'arborescence donne des plus courts chemins. Complexité $O(n^2)$ (matrice d'adjacence).

Question 8

```
void _residue(void) {
    int v = H.n + 1;              /* on part du puits */
    while (v != 0) {              /* jusqu'à la source */
        int u = A[v];             /* arc (u,v) du chemin alpha */
        nw_remove(u, v);          /* une occurrence de (u,v) en moins */
        nw_add(v, u);             /* une occurrence de l'arc opposé en plus */
        v = u;
    }
}
```

On parcourt le chemin α à l'envers ; chaque arc est traité une fois, comme dans la définition du réseau résiduel.

Question 9

Idée : on enregistre, pour chaque arc $e = (u, v) \in F$, un entier $f(e)$ (combien de fois e est utilisé « net »), tel que le réseau courant soit

$$\mu_{\text{cour}}(u, v) = \mu(u, v) - f(u, v) + f(v, u)$$

(on retire les copies utilisées et on ajoute les copies retournées). Au départ $f = 0$.

Mise à jour quand on applique `_residue` avec un chemin α : pour chaque arc (u, v) de α, si $f(v, u) > 0$ on fait $f(v, u) \leftarrow f(v, u) - 1$ (on annule un passage en sens inverse), sinon $f(u, v) \leftarrow$

$f(u,v)+1$. Dans les deux cas, $\mu_{\text{cour}}(u,v)$ baisse de 1 et $\mu_{\text{cour}}(v,u)$ monte de 1 : c'est exactement l'effet de `_residue`. Comme (u,v) existe dans le réseau courant, on garde $0 \leqslant f \leqslant \mu$.

Invariant après j appels : $0 \leqslant f \leqslant \mu$; en tout sommet autre que s et t, autant d'unités de f entrent que sortent (un chemin qui entre dans un sommet en ressort) ; il sort j unités nettes de s.

Décomposition : on extrait un chemin de s à t formé d'arcs où $f > 0$ (en partant de s et en suivant des arcs « chargés », la conservation garantit qu'on ne reste pas bloqué avant t) ; on enlève 1 sur ses arcs et on recommence. On obtient k chemins Π qui utilisent chaque arc e exactement $f(e) \leqslant \mu(e)$ fois : Π est réalisable et $\mathrm{Res}(H,\Pi)$ a pour multiplicités $\mu - f(u,v) + f(v,u)$, c'est-à-dire le réseau obtenu.

(S'il reste après extraction des circuits d'unités de flot sans lien avec s, on les traite comme des passages supplémentaires sur des arcs déjà comptés ; ils ne changent ni k ni les raisonnements qui suivent, qui ne portent que sur les arcs qui sortent ou entrent dans un entourage.)

Question 10

```
void nw_disconnect(void) {
    while (_bfs_tree())           /* tant que t est accessible depuis s */
        _residue();               /* résiduel le long d'un plus court chemin */
}
```

Le dernier appel à `_bfs_tree` a renvoyé `false` : `A` contient alors une arborescence de racine s dans le réseau résiduel final, qui n'atteint pas t, comme demandé.

Question 11

Après k tours, par Q9, le réseau est $\mathrm{Res}(H,\Pi)$ pour un ensemble réalisable Π de k chemins. Par Q12 appliquée à l'entourage $S = \{s\}$ (qui contient s et pas t) :

$$k \leqslant \mu(\partial\{s\}) = \sum_{v} \mu(s,v) \qquad \text{(multiplicités du réseau de départ).}$$

Variant : $\mu(\partial\{s\}) - k$ est un entier naturel qui diminue de 1 à chaque tour. La boucle fait donc au plus $\mu(\partial\{s\})$ tours : elle termine.

Question 12

Chaque chemin π_j commence en $s \in S$ et finit en $t \notin S$: il contient au moins un arc qui sort de S, c'est-à-dire un arc de ∂S (le premier arc dont l'origine est dans S et l'extrémité hors de S). En comptant ces passages :

$$k \leqslant \sum_{e \in \partial S} \mathrm{card}\{j : e \in \pi_j\} \leqslant \sum_{e \in \partial S} \mu(e) = \mu(\partial S),$$

la seconde inégalité venant de la réalisabilité.

Question 13

Soit $e = (u, v) \in F$ avec $u \in S_0$, $v \notin S_0$. Dans $\mathrm{Res}(H, \Pi_0)$, la multiplicité de (u, v) vaut $\mu(e) -$ (utilisations de e) + (utilisations de (v, u)) $\geqslant \mu(e) -$ (utilisations de e).

Si une occurrence de e n'était pas utilisée, cette multiplicité serait $\geqslant 1$: l'arc (u, v) existerait dans le résiduel, et v serait accessible depuis s (puisque u l'est), donc $v \in S_0$: contradiction. Toutes les $\mu(e)$ occurrences sont donc utilisées.

Question 14

Soit $(v, u) \in F$ avec $v \notin S_0$, $u \in S_0$. Dans le résiduel, la multiplicité de (u, v) vaut $\mu(u, v) -$ (utilisations de (u, v)) + (utilisations de (v, u)), avec $\mu(u, v) = 0$ si $(u, v) \notin F$.

Par Q13, si $(u, v) \in F$, toutes ses occurrences sont utilisées : les deux premiers termes se compensent. Il reste la multiplicité « utilisations de (v, u) ». Si elle était $\geqslant 1$, (u, v) serait un arc du résiduel et v serait accessible : contradiction. Donc (v, u) n'est jamais utilisé.

Question 15

Un chemin de $s \in S_0$ à $t \notin S_0$ sort de S_0 une fois de plus qu'il n'y entre. D'après Q14, aucun chemin n'entre dans S_0 : chaque chemin sort donc de S_0 **exactement une fois**. Le nombre total d'utilisations d'arcs de ∂S_0 vaut donc k_0.

D'après Q13, ce même nombre vaut $\sum_{e \in \partial S_0} \mu(e) = \mu(\partial S_0)$ (toutes les occurrences sont utilisées). D'où

$$k_0 = \mu(\partial S_0).$$

Question 16

Par Q12, $k_0 \leqslant \mu(\partial S)$ pour tout entourage S, donc $k_0 \leqslant \sigma(H)$. Par Q15, $\sigma(H) \leqslant \mu(\partial S_0) = k_0$. D'où

$$k_0 = \sigma(H) = \mu(\partial S_0).$$

Après `nw_disconnect` : par Q9, le réseau obtenu est $\mathrm{Res}(H, \Pi_0)$ pour un Π_0 réalisable, et il n'a plus de chemin de s à t. Le dernier parcours en largeur a marqué ($A[v] \geqslant 0$) exactement les sommets accessibles depuis s, soit S_0. C'est un entourage (s marqué, t non) de multiplicité $\mu(\partial S_0) = \sigma(H)$: un entourage de plus petite bordure.

Question 17

```
graph *gr_create(unsigned int n) {
    graph *G = malloc(sizeof(graph));
    G->n = n;
    G->neighbours = malloc((n + 1) * sizeof(link *));
    for (unsigned int v = 0; v <= n; v++)
        G->neighbours[v] = NULL;      /* liste d'adjacence vide */
```

```
    return G;
}
```

Question 18

```
/* insère v en tête de la liste d'adjacence de u */
static void push_link(graph *G, int u, int v) {
    link *l = malloc(sizeof(link));
    l->node = v;
    l->prev = NULL;
    l->next = G->neighbours[u];
    if (l->next != NULL) l->next->prev = l;
    G->neighbours[u] = l;
}

void gr_add(graph *G, int u, int v) {
    push_link(G, u, v);    /* arc u -> v */
    push_link(G, v, u);    /* arc v -> u : l'arête est stockée deux fois */
}
```

Insertion en tête en $O(1)$, sans oublier le chaînage arrière (`prev`).

Question 19

```
int degree(graph *G, int v) {
    int d = 0;
    for (link *l = G->neighbours[v]; l != NULL; l = l->next)
        d++;
    return d;
}
```

Question 20

```
int edges(graph *G) {
    int somme = 0;
    for (unsigned int v = 1; v <= G->n; v++)
        somme += degree(G, v);
    return somme / 2;      /* chaque arête est stockée comme deux arcs */
}
```

Question 21

```
double density(graph *G) {
    return (double) edges(G) / G->n;
}
```

La conversion en `double` évite la division entière.

Question 22

```
void reduce(graph *G, int r) {
    unsigned int n = G->n;
    unsigned int m = edges(G);
    nw_init(n);                                  /* n + 2 sommets, aucun arc */
    for (unsigned int u = 1; u <= n; u++) {
        H.mu[0][u] = m * n * n;                  /* arc (s, u) */
        for (link *l = G->neighbours[u]; l != NULL; l = l->next)
            H.mu[u][l->node] = n * n;            /* arc (u, v) pour l'arête {u,v} */
        H.mu[u][n + 1] = m * n * n - n * n * degree(G, u) + 2 * r;   /* arc (u, t) */
    }
}
```

Chaque arête $\{u, v\}$ apparaît dans la liste de u (donne l'arc (u, v)) et dans celle de v (donne (v, u)). La multiplicité de (u, t) vaut $n^2(m - \deg u) + 2r \geqslant 0$; si elle est nulle, il n'y a simplement pas d'arc.

Question 23

Faisons la somme $\sum_{v\in X} \deg_G(v)$: chaque arête $\{u, v\}$ de G y compte autant de fois qu'elle a d'extrémités dans X :

- deux fois si $u, v \in X$ (arête de E_X) ;
- une fois si une seule extrémité est dans X (arête « coupée ») ;
- zéro fois sinon.

$$\sum_{v\in X} \deg_G(v) = 2|E_X| + c_X, \qquad c_X = \operatorname{card}\{\{u, v\} \in E : u \in X,\ v \notin X\}.$$

On isole $|E_X|$ et on divise par $|X|$:

$$\delta(G_X) = \frac{|E_X|}{|X|} = \frac{\sum_{v\in X} \deg_G(v) - c_X}{2|X|}.$$

Question 24

Les arcs qui sortent de $S = \{s\} \cup X$ sont :

- (s, v) pour $v \notin X$: $(n - |X|)$ arcs de multiplicité mn^2 ;
- (u, t) pour $u \in X$: multiplicité $mn^2 - n^2 \deg_G(u) + 2r$ chacun ;
- (u, v) avec $u \in X$, $v \notin X$, $\{u, v\} \in E$: c_X arcs de multiplicité n^2.

$$\mu(\partial S) = (n-|X|)mn^2 + |X|mn^2 - n^2 \sum_{u \in X} \deg_G(u) + 2r|X| + n^2 c_X = mn^3 + 2r|X| - n^2 \Big(\sum_{u \in X} \deg_G(u) - c_X \Big).$$

Avec Q23, la parenthèse vaut $2|X|\delta(G_X)$:

$$\boxed{\mu(\partial S) = mn^3 + 2|X|(r - n^2\delta(G_X)).}$$

Pour $S = \{s\}$: seuls les n arcs (s, v) sortent, d'où $\mu(\partial\{s\}) = n \cdot mn^2 = mn^3$.

Question 25

Un entourage contient s, pas t : c'est $\{s\} \cup X$ pour un $X \subseteq V$. Par Q24, sa bordure vaut mn^3 si $X = \emptyset$, et $mn^3 + 2|X|(r - n^2\delta(G_X))$ sinon. Donc

$$\sigma(H) = \min \Big(mn^3,\ \min_{X \neq \emptyset} \big[mn^3 + 2|X|(r - n^2\delta(G_X)) \big] \Big) \leqslant mn^3.$$

- $\sigma(H) < mn^3 \iff$ il existe $X \neq \emptyset$ avec $r - n^2\delta(G_X) < 0$, c'est-à-dire $\delta(G_X) > \frac{r}{n^2} \iff \rho(G) > \frac{r}{n^2}$.
- Comme toujours $\sigma(H) \leqslant mn^3$, la négation donne $\sigma(H) = mn^3 \iff \rho(G) \leqslant \frac{r}{n^2}$.

Question 26

```
bool oracle(graph *G, int r) {
    reduce(G, r);
    nw_disconnect();                          /* A : sommets accessibles = S0 */
    for (unsigned int v = 1; v <= G->n; v++)
        if (A[v] >= 0) return true;           /* S0 contient un sommet du graphe */
    return false;                             /* S0 = {s} */
}
```

Principe : après `nw_disconnect`, $S_0 = \{v : A[v] \geqslant 0\} = \{s\} \cup X_0$ est un entourage de plus petite bordure (Q16).

- Si $\rho(G) > \frac{r}{n^2}$: $\sigma(H) < mn^3$ (Q25), donc $X_0 \neq \emptyset$ (sinon la bordure vaudrait mn^3) : on renvoie `true`.

- Si $\rho(G) \leqslant \frac{r}{n^2}$: $\{s\}$ est lui-même de plus petite bordure. Or S_0 est inclus dans **tout** entourage minimal S : pour un tel S, l'argument de Q15 montre que les arcs qui sortent de S sont saturés et que ceux qui y entrent ne sont pas utilisés, donc aucun arc du résiduel ne sort de S, et les sommets accessibles depuis s restent dans S. Avec $S = \{s\}$: $S_0 = \{s\}$, on renvoie `false`.

Bonus : quand la réponse est `true`, la ligne $\mu(\partial S_0) < mn^3$ de Q24 montre que $\delta(G_{X_0}) > \frac{r}{n^2}$: `A` désigne un sous-graphe témoin.

Question 27

$$\left|\frac{m_1}{n_1} - \frac{m_2}{n_2}\right| = \frac{|m_1 n_2 - m_2 n_1|}{n_1 n_2}.$$

Le numérateur est un entier, non nul puisque les fractions diffèrent : il vaut au moins 1. Le dénominateur vérifie $n_1 n_2 \leqslant n^2$. D'où

$$\left|\frac{m_1}{n_1} - \frac{m_2}{n_2}\right| \geqslant \frac{1}{n^2}.$$

Question 28

Soit Y réalisant $\rho(G) = \delta(G_Y)$. Par définition, $\rho(G) \geqslant \delta(G_X)$. Les deux densités sont de la forme $\frac{m'}{n'}$ avec $(m', n') \in \Delta$ (un sous-graphe induit a au plus m arêtes et entre 1 et n sommets).

Si $\rho(G) \neq \delta(G_X)$, Q27 donne $\rho(G) \geqslant \delta(G_X) + \frac{1}{n^2}$: G_Y serait un sous-graphe induit de densité $\geqslant \delta(G_X) + \frac{1}{n^2}$, ce que l'hypothèse exclut. Donc $\delta(G_X) = \rho(G)$.

Question 29

Encadrement : $0 \leqslant \rho(G) \leqslant \frac{n-1}{2}$ (un sommet a au plus $n-1$ voisins, donc $m' \leqslant \frac{n'(n'-1)}{2}$). Avec $r_{\max} = \frac{n^2(n-1)}{2}$ (entier), l'oracle répond `false` en $r_{\max}$. La réponse de l'oracle est décroissante en r (vraie pour $r < n^2\rho$, fausse ensuite).

```
void binary_search(graph *G) {
    int n = G->n;
    int lo = -1;                        /* oracle(lo) vrai (r = -1 : convention) */
    int hi = n * n * (n - 1) / 2;       /* oracle(hi) faux */
    while (hi - lo > 1) {
        int mid = (lo + hi) / 2;
        if (oracle(G, mid)) lo = mid; else hi = mid;
    }
    if (lo == -1) {                     /* rho(G) = 0 : pas d'arête */
        _arr_init(A); A[0] = 0; A[1] = 0;      /* un sommet seul convient */
    } else
        oracle(G, lo);                  /* A décrit X avec delta(G_X) > lo / n^2 */
}
```

Correction : à la fin, $hi = lo + 1$, $\rho(G) > \frac{lo}{n^2}$ et $\rho(G) \leqslant \frac{lo+1}{n^2}$. Le dernier appel `oracle(G, lo)` laisse dans `A` un ensemble X avec $\delta(G_X) > \frac{lo}{n^2}$ (bonus de Q26). Si $\delta(G_X) \neq \rho(G)$, Q27 donnerait

$\delta(G_X) \leqslant \rho(G) - \frac{1}{n^2} \leqslant \frac{lo}{n^2}$: contradiction. Donc $\delta(G_X) = \rho(G)$.

Efficacité : $O(\log n^3) = O(\log n)$ appels à l'oracle, au lieu de tester toutes les valeurs de r.

Question 30

- `densest_subgraph.c` commence par `#include "network.h"` pour connaître le type `network` et les fonctions `nw_init`, `nw_disconnect`, `_arr_init`.
- Les variables globales `H` et `A` sont **définies** dans `network.c` ; pour les utiliser ailleurs, il faut les **déclarer** : `extern network H; extern int A[NMAX];` (idéalement ajoutées dans `network.h`).
- On compile chaque fichier en objet puis on lie : `gcc -c network.c`, `gcc -c densest_subgraph.c`, `gcc network.o densest_subgraph.o -o densest`.

Question 31

Chaque arête est orientée dans un seul sens, et chaque arc compte pour exactement un degré entrant :

$$\sum_{v \in V} \deg^-_{\widehat{G}}(v) = m.$$

Une moyenne est inférieure au maximum :

$$\delta(G) = \frac{m}{n} = \frac{1}{n} \sum_{v} \deg^-_{\widehat{G}}(v) \leqslant \max_{v} \deg^-_{\widehat{G}}(v).$$

Question 32

Degré entrant de v_i : l'arête $\{v_i, v_{i'}\}$ est orientée vers $v_{\max(i,i')}$. Les arcs qui entrent dans v_i viennent donc des arêtes $\{v_i, v_{i'}\}$ avec $i' < i$, c'est-à-dire des voisins de v_i encore présents dans G_i (qui contient $v_1, \dots, v_i$) :

$$\deg^-_{\widehat{G}_0}(v_i) = \deg_{G_i}(v_i).$$

Comme v_i minimise le degré dans G_i, son degré est au plus la moyenne : $\deg_{G_i}(v_i) \leqslant \frac{2m_i}{i} = 2\delta(G_i)$, où m_i est le nombre d'arêtes de G_i. (Pour $i = 1$, G_1 n'a pas d'arête et les deux membres sont nuls.)

$$\deg^-_{\widehat{G}_0}(v_i) \leqslant 2\delta(G_i) \leqslant 2\delta(\Gamma).$$

Facteur : soit X avec $\delta(G_X) = \rho(G)$. En gardant de $\widehat{G}_0$ les arcs entre sommets de X, on obtient une orientation de G_X dont les degrés entrants sont plus petits. Par Q31 :

$$\rho(G) = \delta(G_X) \leqslant \max_{v \in X} \deg^-_{\widehat{G}_0}(v) \leqslant 2\delta(\Gamma).$$

L'algorithme renvoie Γ avec $\delta(\Gamma) \geqslant \frac{1}{2}\rho(G)$: c'est une **2-approximation**.

Question 33

Données :

- `deg[v]` : degré courant de v dans G_i ; `removed[v]` : booléen ;
- `bucket[d]` pour $d \in [\![0, n]\!]$: liste doublement chaînée des sommets restants de degré d, et pour chaque sommet un pointeur vers son maillon (retrait en $O(1)$) ;
- `dmin` : un indice au plus égal au plus petit degré présent ;
- `m_cur` : nombre d'arêtes de G_i ; `order[]` : ordre de suppression ; `best_i` : l'indice i de la meilleure densité vue.

Étape i (i de n à 2) :

1. calculer $\delta(G_i) =$ `m_cur / i` en $O(1)$ et mettre à jour `best_i` ;
2. tant que `bucket[dmin]` est vide, faire `dmin++` ; prendre v dans ce seau et le retirer ;
3. marquer v supprimé et parcourir sa liste d'adjacence : pour chaque voisin u non supprimé, déplacer u de `bucket[deg[u]]` vers `bucket[deg[u]-1]` en $O(1)$, faire `deg[u]--`, `m_cur--`, et `dmin = min(dmin, deg[u])`.

À la fin, $\Gamma = G_{\text{best_i}}$: ce sont les sommets qui n'étaient pas supprimés avant l'étape `best_i`, qu'on lit dans `order`.

Complexité : initialisation des degrés et des seaux $O(n+m)$. Chaque sommet est retiré une fois et sa liste parcourue une fois : $O(n+m)$ au total. `dmin` augmente d'au plus n au total, plus 1 pour chaque fois qu'il a diminué ; il diminue d'au plus 1 par arête retirée. Donc le total de ses incréments est $O(n+m)$.

Temps total : $O(n+m)$.

Le piège

1. En Q14, oublier qu'utiliser l'arc (v, u) ajoute une copie de (u, v) dans le résiduel : c'est elle qui rendrait v accessible.
2. En Q26, répondre `true` dès que $\sigma(H) \leqslant mn^3$: il faut un X non vide, et c'est la minimalité de l'ensemble accessible S_0 qui garantit $S_0 = \{s\}$ dans le cas d'égalité.
3. En Q33, rechercher le degré minimal par un parcours de tous les sommets à chaque étape : on retombe en $O(n^2)$.

La méthode

1. Ford-Fulkerson : augmenter tant qu'un chemin existe dans le résiduel ; à la fin, les sommets accessibles forment une coupe minimale (arcs sortants saturés, arcs entrants inutilisés).
2. Réduction à une coupe minimale : construire un réseau dont la bordure de $\{s\} \cup X$ s'exprime avec la quantité à optimiser, puis dichotomie sur un seuil entier grâce à l'écart minimal entre fractions.
3. Approximation : relier la solution gloutonne à une borne de l'optimum (ici par une orientation et le degré entrant maximal). Linéarité : des seaux indexés par le degré.

Rappels de cours utilisés

- **Garde d'inclusion.** `#ifndef X` / `#define X` / `#endif` : le contenu n'est inclus qu'une fois par fichier compilé, même si l'en-tête est inclus plusieurs fois (directement ou par d'autres en-têtes).
- **Compilation séparée en C.** Chaque fichier `.c` inclut les en-têtes `.h` dont il a besoin et se compile en un fichier objet (`gcc -c f.c`). L'édition de liens assemble les objets (`gcc f.o g.o -o prog`). Une variable globale définie dans un fichier s'utilise ailleurs grâce à une déclaration `extern`.
- **Parcours en largeur.** File FIFO : on défile un sommet, on enfile ses voisins non encore vus en notant leur prédécesseur. Chaque sommet est enfilé au plus une fois ; l'arborescence obtenue donne des plus courts chemins en nombre d'arcs.
- **Coupe et chemins (Menger, Ford-Fulkerson).** Le nombre maximal de chemins réalisables de s à t est égal à la multiplicité minimale d'une bordure séparant s de t. Les chemins augmentants dans le réseau résiduel permettent de les construire.